ASTRA RUNTIME

Capability-Based Cybersecurity-Hardened
Userspace Runtime for Linux

This Document Comprises of everything that has been Built up until now

Version 0.1.0 | March 17, 2026

Team Lead: Shiva | Module: M-01 Core Runtime

Table of Contents

1. Project Overview 3

2. Architecture & Design Philosophy 4

3. Naming Convention 5

4. File Structure 6

5. Foundation Types (types.h) 7

6. Error Handling (result.h) 8

7. Capability Token System (capability.h) 10

8. Service Interface & Registry (service.h) 14

9. Event Bus (event_bus.h) 18

10. Process Manager (process.h) 22

11. Core Runtime (runtime.h) 26

12. Logger (logger.h) 28

13. Demo Service (demo_service.h) 31

14. Build Instructions 33

15. Module Roadmap & Team Assignments 34

1. Project Overview

Astra Runtime is a modular, capability-based, cybersecurity-hardened userspace runtime for Linux. It operates as a "userspace kernel" on top of the Linux kernel, providing a microkernel-style architecture where every module runs as an isolated service. All inter-module communication is mediated through capability-gated IPC channels, and every resource access requires an unforgeable token.

The project is built with C++23, targets x86-64 Linux exclusively, and enforces strict security hardening at both compile time (-fstack-protector-strong, -D_FORTIFY_SOURCE=2, -fPIE) and link time (-pie, -Wl,-z,relro, -Wl,-z,now, -Wl,-z,noexecstack). All warnings are errors (-Werror).

Key Properties

- **Capability-Based Security:** Every resource access is gated by a 128-bit capability token generated from hardware RNG (RDRAND).
- **Process Isolation:** Modules are isolated via Linux namespaces, seccomp-BPF syscall filtering, and cgroups v2 resource limits.
- **Zero-Copy IPC:** All inter-module communication uses shared-memory ring buffers with HMAC-signed messages.
- **Erlang-Style Supervision:** Crashed services are automatically restarted with configurable policies (ALWAYS, ON_FAILURE, EXPONENTIAL backoff).
- **Kernel-Level Observability:** eBPF probes monitor every syscall, IPC message, and capability event in real time with near-zero overhead.
- **Rust-Style Error Handling:** C++23 std::expected<T, Error> for all error paths. No exceptions in hot paths.

2. Architecture & Design Philosophy

Astra is structured as 21 modules across 6 layers. Only the Core module (M-01) is currently implemented. All other modules will integrate through the service interface and capability system defined by M-01.

Layer Architecture

Layer	Modules	Purpose
Layer 1: Core	M-01 Core, M-21 ASM Core	Runtime lifecycle, capability tokens, service registry, event bus, process manager, crypto primitives
Layer 2: Security	M-02 Isolation, M-03 IPC, M-18 Crypto	Namespace/seccomp/cgroup sandboxing, HMAC-signed IPC channels, Ed25519 signatures
Layer 3: Reliability	M-04 Checkpoint, M-05 Replay	Process state snapshots, deterministic replay for debugging
Layer 4: Performance	M-06 Allocator, M-07 Job Engine, M-17 Profiler	Custom memory allocator, task scheduler, performance metrics
Layer 5: Intelligence	M-08 AI, M-09 eBPF, M-11 AQL	Anomaly detection, kernel-level observability, query language
Layer 6: Platform	M-10 WASM, M-13 Net, M-14 VFS, M-16 HAL, M-20 Loader	WebAssembly sandbox, networking, virtual filesystem, hardware abstraction

Core Subsystem Interaction

The Runtime object owns all four core subsystems. Modules access them through typed accessors: runtime.capabilities(), runtime.services(), runtime.events(), and runtime.processes(). The boot sequence initialises subsystems in dependency order, and shutdown tears them down in reverse.

```
Runtime::init(config)
  1. Platform checks (CPU features, compiler, OS)
  2. ASM Core self-test (M-21 crypto primitives)
  3. CapabilityManager::init()  -> token pool allocated
```

```
  4. EventBus::init()           -> ring buffer allocated
  5. ServiceRegistry::init()    -> service slots allocated
  6. ProcessManager::init()     -> process pool allocated

Runtime::run()
  -> main event loop: dispatch events, reap children, health checks

Runtime::shutdown()
  6. ProcessManager::shutdown()
  5. ServiceRegistry::shutdown() (reverse order: stopAll -> unregister)
  4. EventBus::shutdown()
  3. CapabilityManager::shutdown() (secure wipe all tokens)
  2. ASM Core cleanup
  1. Platform cleanup
```

3. Naming Convention

Astra uses a strict Hungarian-style naming convention across all code. Every variable name encodes its scope and type for instant readability.

Scope Prefixes

Prefix	Scope	Example
m_	Class member variable	m_uMaxTokens, m_bInitialised
l	Local variable	lUIdx, lSzName, lBFound
a	Function argument/parameter	aUMaxTokens, aSzFilePath
g_	Global variable	g_logCore, g_logIpc

Type Prefixes (after scope prefix)

Prefix	Type	Example
i	int (signed integer)	m_iExitCode, aILine
u	unsigned integer	m_uMaxTokens, lUIdx
b	bool	m_bInitialised, aBAlsoConsole
sz	string (std::string or C-string)	m_szModuleName, aSzFilePath
e	enum	m_eState, aELevel
p	pointer	m_pTokenPool, aPService
v	vector	lVThreads
arr	array (std::array or C-array)	m_arrUId, m_arrSubscribers
fn	function/callback	m_fnTokenEvent, m_fnIsolationHook
ofs	output file stream	m_ofsFile
mtx	mutex	m_mtxWrite
cap	capability token	m_capToken

Examples

```
// Member: m_ + type prefix + descriptive name
U32                 m_uMaxTokens;       // m_ = member, u = unsigned
bool                m_bInitialised;     // m_ = member, b = bool
std::string         m_szModuleName;     // m_ = member, sz = string
CapabilityToken     m_capToken;         // m_ = member, cap = capability
std::atomic<bool>   m_bSpinlock;        // m_ = member, b = bool

// Local: l + type prefix + descriptive name
U32 lUIdx = 0;                          // l = local, U = unsigned
std::string lSzTimestamp = getTimestamp(); // l = local, Sz = string

// Argument: a + type prefix + descriptive name
bool initialise(std::string_view aSzModuleName, LogLevel aEMinLevel);
```

4. File Structure

Headers (include/astra/)

File	Contains	Purpose
common/types.h	U8..U64, I8..I64, ModuleId, ProcessId, ServiceId, ChannelId	Foundation type aliases used everywhere
common/result.h	Error, ErrorCode, ErrorCategory, Result<T>, Status	Error handling: C++23 std::expected wrapper
core/capability.h	Permission, CapabilityToken, CapabilityManager	Capability token system (create, derive, validate, revoke)
core/service.h	ServiceState, IService, ServiceHandle, ServiceRegistry	Service interface and registry
core/event_bus.h	EventType, Event, EventHandler, EventBus	Publish/subscribe event notification
core/process.h	ProcessState, RestartPolicy, IsolationProfile, ProcessConfig, ProcessDescriptor, ProcessManager	Process lifecycle and supervision
core/runtime.h	RuntimeConfig, RuntimeState, Runtime	Central orchestrator owning all subsystems
core/logger.h	LogLevel, Logger, LOG_* macros	Per-module file-backed logger
core/demo_service.h	DemoService	Teaching example of IService implementation

Sources (src/)

File	Implements	Global Logger Defined
core/runtime.cpp	Runtime lifecycle, init stages, main loop	g_logCore
core/capability.cpp	CapabilityManager: create, derive, validate, revoke	g_logCapability
core/service_registry.cpp	ServiceRegistry: register, lookup, start/stop	g_logSvcRegistry
core/event_bus.cpp	EventBus: publish, subscribe, dispatch	g_logEventBus
core/process_manager.cpp	ProcessManager: spawn, kill, suspend, reap	g_logProcMgr
core/demo_service.cpp	DemoService: 5 interactive demos	g_logDemo
core/logger.cpp	Logger: file I/O, timestamps, formatting	(implementation)
main.cpp	Main entry point: signal handlers, CLI parsing	g_logMain
demo_main.cpp	Demo entry point: boots runtime, runs demo	g_logDemoMain

5. Foundation Types (common/types.h)

File: include/astra/common/types.h

Namespace: astra

Defines fixed-width type aliases to prevent platform-dependent size surprises in security-critical code. All modules must use these instead of raw int/unsigned.

Type Aliases

Alias	Underlying Type	Usage
U8	std::uint8_t	Log levels, service states, process states
U16	std::uint16_t	Event types, module IDs
U32	std::uint32_t	Service IDs, ring sizes, counters
U64	std::uint64_t	Process IDs, capability IDs, permission bitmasks, timestamps
I8 / I16 / I32 / I64	Signed equivalents	Exit codes, signal numbers
SizeT	std::size_t	Buffer sizes, array lengths

Identifier Types

Type	Alias For	What It Identifies
ProcessId	U64	Astra-internal process ID (not Linux PID)
CapabilityId	U64	Capability token identifier
ServiceId	U32	Registered service slot (1-based, 0 = invalid)
ChannelId	U32	IPC channel number (future M-03)

ModuleId Enum

Matches the M-XX numbering from the project specification. Used for event source tagging and service registration.

Value	Module	Value	Module
CORE = 1	M-01 Core Runtime	EBPF = 9	M-09 Observability
ISOLATION = 2	M-02 Isolation	WASM = 10	M-10 WebAssembly
IPC = 3	M-03 IPC	AQL = 11	M-11 Query Language
CHECKPOINT = 4	M-04 Checkpoint	VERIFY = 12	M-12 Verification
REPLAY = 5	M-05 Replay	NET = 13	M-13 Networking
ALLOCATOR = 6	M-06 Memory	VFS = 14	M-14 Virtual FS
JOB_ENGINE = 7	M-07 Scheduler	HAL = 16	M-16 Hardware Abstraction
AI = 8	M-08 AI Engine	ASM_CORE = 21	M-21 Assembly Crypto

6. Error Handling (common/result.h)

File: include/astra/common/result.h

Namespace: astra

Astra uses C++23 std::expected<T, Error> instead of exceptions. Every function that can fail returns either a Result<T> (value or error) or a Status (void or error). This is similar to Rust's Result<T, E> and ensures errors cannot be silently ignored.

Core Types

Type	Definition	Use Case
Result<T>	std::expected<T, Error>	Functions returning a value on success
Status	std::expected<void, Error>	Functions returning nothing on success
Error	Class with code + category + message	Carries error details across boundaries

How to Use Result<T>

```
// Returning success:
Result<CapabilityToken> create(Permission perms, U64 owner)
{
    CapabilityToken token = ...;
    return token;  // implicitly wraps in expected
}

// Returning failure:
return std::unexpected(makeError(
    ErrorCode::PERMISSION_DENIED,
    ErrorCategory::CORE,
    "Child permissions exceed parent"
));

// Checking the result:
auto result = manager.create(Permission::IPC_SEND, ownerId);
if (result.has_value())
{
```

```
    CapabilityToken token = result.value();  // success
}
else
{
    Error err = result.error();  // failure
    // err.code(), err.category(), err.message()
}
```

Error Codes

Code	Value	Meaning
OK	0	No error
INVALID_ARGUMENT	1	Bad parameter passed to function
OUT_OF_MEMORY	2	Allocation failed (pool exhausted)
NOT_INITIALIZED	3	Subsystem not yet initialised
ALREADY_INITIALIZED	4	Double init attempt
NOT_FOUND	5	Service/process/token not in registry
ALREADY_EXISTS	6	Duplicate registration attempt
PERMISSION_DENIED	7	Capability check failed
RESOURCE_EXHAUSTED	11	Token pool or process pool full
CAPABILITY_INVALID	50	Token is null or malformed
CAPABILITY_REVOKED	52	Token was revoked
HMAC_VERIFICATION_FAIL	54	IPC message HMAC mismatch
ANOMALY_DETECTED	58	eBPF anomaly detector triggered

7. Capability Token System (capability.h)

File: include/astra/core/capability.h | Implementation: src/core/capability.cpp

Namespace: astra::core

The capability model is the central security primitive of Astra. Every resource access, service call, and IPC message is gated by a capability token. Tokens are unforgeable 128-bit values generated from Intel RDRAND with embedded permission bitmasks.

7.1 Permission Enum

Type: enum class Permission : U64

A 64-bit bitmask where each bit represents a specific permission. Permissions are composed with bitwise OR and checked with bitwise AND.

Permission	Bit	What It Allows
SVC_REGISTER	1 << 0	Register a service with the registry
SVC_LOOKUP	1 << 1	Look up service handles by name/ID
SVC_UNREGISTER	1 << 2	Remove a service from the registry
IPC_SEND	1 << 8	Send messages on an IPC channel
IPC_RECV	1 << 9	Receive messages from an IPC channel
IPC_CREATE_CHAN	1 << 10	Create new IPC channels
PROC_SPAWN	1 << 16	Spawn new supervised
PROC_KILL	1 << 17	Send signals to processes
PROC_INSPECT	1 << 18	Read process state (for checkpointing)
MEM_ALLOC	1 << 24	Allocate from the runtime allocator
MEM_MAP	1 << 25	Map shared memory regions
FS_READ	1 << 32	Read from virtual filesystem
FS_WRITE	1 << 33	Write to virtual filesystem
FS_EXEC	1 << 34	Execute from virtual filesystem

NET_CONNECT	1 << 40	Outbound network connections
NET_LISTEN	1 << 41	Listen for inbound connections
NET_RAW	1 << 42	Raw socket access
SYS_ADMIN	1 << 56	Full administrative access (root token only)
SYS_AUDIT	1 << 57	Read audit/security logs
SYS_MODULE_LOAD	1 << 58	Load dynamic modules at runtime
SYS_SHUTDOWN	1 << 59	Request runtime shutdown

Permission Helper Functions

Function	Signature	Description
operator|	Permission | Permission -> Permission	Combine permissions: SVC_REGISTER | IPC_SEND
operator&	Permission & Permission -> Permission	Intersect permissions: token.perms & required
operator~	~Permission -> Permission	Invert a permission set
hasPermission	hasPermission(set, required) -> bool	Check if set contains all bits of required

7.2 CapabilityToken Struct

Field	Type	Description
m_arrUId	std::array<U64, 2>	128-bit unique identifier (two U64s from RDRAND)
m_ePermissions	Permission	64-bit permission bitmask
m_uEpoch	U64	Epoch number for revocation tracking
m_uOwnerId	U64	ProcessId or ServiceId of the token holder
m_arrUParentId	std::array<U64, 2>	Parent token ID ({0,0} = root token)

CapabilityToken Methods

Method	Returns	Description
isValid()	bool	True if token ID is non-zero (not a null token)
null()	CapabilityToken	Static: returns a null/empty token
operator==(other)	bool	Compare by token ID only (not permissions)

7.3 CapabilityManager Class

The sole authority for token lifecycle. Owns the pre-allocated token pool.

Public Methods

Method	Signature	Description
init	Status init(U32 maxTokens = 4096)	Allocate token pool. Call once at startup.
shutdown	void shutdown()	Secure-wipe and deallocate all tokens.
create	Result<CapabilityToken> create(Permission perms, U64 ownerId)	Create a new root token. Generates 128-bit ID via RDRAND.
derive	Result<CapabilityToken> derive(parent, childPerms, newOwnerId)	Create a child token. childPerms must be subset of parent (monotonicity).
validate	bool validate(token, requiredPerms)	Check token exists, not revoked, has required permissions. Lock-free fast path.
revoke	Result<U32> revoke(token)	Revoke token + all descendants. Returns count revoked.
currentEpoch	U64 currentEpoch()	Get global epoch counter (incremented on each revocation).
activeTokenCount	U32 activeTokenCount()	Number of non-revoked tokens in the pool.
setTokenEventCallback	void setTokenEventCallback(cb)	Register callback for token lifecycle events (for M-09 eBPF).

Thread Safety Model

- **Spinlock (m_bSpinlock):** Used ONLY for create(), derive(), revoke(). These are rare operations.
- **Lock-Free Validation:** validate() is the hot path (called on every IPC message). Uses atomic epoch read + lock-free pool scan. No spinlock, no mutex, no contention.
- **Uniqueness via RDRAND:** Token IDs generated from Intel RDRAND instruction (hardware entropy). 128-bit keyspace makes collisions impossible (probability 1 in 2^128).

How to Use (Code Example)

```
// Create a root token with IPC permissions
```

```
auto result = runtime.capabilities().create(
    Permission::IPC_SEND | Permission::IPC_RECV,
    myServiceId
);
CapabilityToken myToken = result.value();

// Derive a restricted child token (send-only)
auto childResult = runtime.capabilities().derive(
    myToken,                // parent
    Permission::IPC_SEND,   // subset of parent's permissions
    childServiceId          // new owner
);

// Validate before performing an operation
if (runtime.capabilities().validate(someToken, Permission::IPC_SEND))
{
    // Token is valid and has IPC_SEND permission
    sendMessage(channel, payload);
}

// Revoke a compromised token (cascades to all children)
auto revokeResult = runtime.capabilities().revoke(compromisedToken);
U32 revokedCount = revokeResult.value();  // includes all descendants
```

8. Service Interface & Registry (service.h)

File: include/astra/core/service.h | Implementation: src/core/service_registry.cpp

Namespace: astra::core

8.1 ServiceState Enum

State	Value	Meaning
UNREGISTERED	0	Slot is free
REGISTERED	1	Registered but not yet started
STARTING	2	onInit() has been called
RUNNING	3	Fully operational
STOPPING	4	Shutdown in progress
STOPPED	5	Cleanly stopped
FAILED	6	Crashed or failed to start

8.2 IService Interface

Every module must implement this interface to integrate with the Core Runtime.

Method	Signature	What to Do Here
name()	string_view name() const	Return a unique human-readable name, e.g. "ipc-channel-manager"
moduleId()	ModuleId moduleId() const	Return your module ID, e.g. ModuleId::IPC
onInit()	Status onInit()	Allocate resources, open files, initialise internal state
onStart()	Status onStart()	Begin main work. Called when runtime enters RUNNING state.
onStop()	Status onStop()	Release resources, flush state. Must complete within timeout.
isHealthy()	bool isHealthy() const	Return false to trigger restart by ProcessManager.

IService Lifecycle

```
register -> onInit() -> onStart() -> [RUNNING] -> onStop() -> unregister
                                          |
                                    isHealthy() called periodically
                                          |
                                    false -> restart via ProcessManager
```

8.3 ServiceHandle Struct

Field	Type	Description
m_uId	ServiceId	Unique service ID (array index + 1, 0 = invalid)
m_eModuleId	ModuleId	Which module this service belongs to
m_szName	std::string	Human-readable name
m_capToken	CapabilityToken	The capability token assigned to this service

8.4 ServiceRegistry Class

Method	Signature	Description
init	Status init(U32 maxServices = 64)	Allocate service slots. Call once.
shutdown	void shutdown()	Stop all services in reverse registration order, deallocate.
registerService	Result<ServiceHandle> registerService(IService*, CapabilityToken)	Register a service. Token must have SVC_REGISTER permission.
unregisterService	Status unregisterService(ServiceId, CapabilityToken)	Remove a service. Requires original registration capability.
lookupById	Result<ServiceHandle> lookupById(ServiceId)	O(1) lookup by ID.
lookupByName	Result<ServiceHandle> lookupByName(string_view)	O(n) scan by name.
lookupByModule	Result<ServiceHandle> lookupByModule(ModuleId)	O(n) scan by module ID.
getServicePtr	IService* getServicePtr(ServiceId)	Direct pointer (internal use only, bypasses IPC).
startAll	Status startAll()	Call onInit() + onStart() on all registered services.
stopAll	Status stopAll()	Call onStop() on all services in reverse order.
registeredCount	U32 registeredCount()	Number of registered services.
runningCount	U32 runningCount()	Number of services in RUNNING state.

How to Register a Service (Code Example)

```
// 1. Derive a token with SVC_REGISTER permission
auto tokenResult = runtime.capabilities().derive(
    rootToken,
    Permission::SVC_REGISTER | Permission::SVC_LOOKUP,
    myOwnerId
);
```

```
CapabilityToken svcToken = tokenResult.value();

// 2. Create your service instance
auto myService = std::make_unique<MyIpcService>(runtime);

// 3. Register with the ServiceRegistry
auto handleResult = runtime.services().registerService(
    myService.get(),
    svcToken
);
ServiceHandle handle = handleResult.value();
// handle.m_uId is now your service ID
```

9. Event Bus (event_bus.h)

File: include/astra/core/event_bus.h | Implementation: src/core/event_bus.cpp

Namespace: astra::core

The event bus provides loose coupling between modules. Publishers emit events without knowing who is listening, and subscribers receive events without knowing who published them. Built on a lock-free ring buffer for minimal latency.

9.1 EventType Enum

Range	Category	Events
1-99	Runtime Lifecycle	RUNTIME_INITIALIZING, RUNTIME_RUNNING, RUNTIME_SHUTTING_DOWN, RUNTIME_STOPPED
100-199	Service Lifecycle	SERVICE_REGISTERED, SERVICE_STARTED, SERVICE_STOPPED, SERVICE_FAILED, SERVICE_UNREGISTERED
200-299	Process Lifecycle	PROCESS_SPAWNED, PROCESS_EXITED, PROCESS_CRASHED, PROCESS_RESTARTED
300-399	Capability Events	CAPABILITY_CREATED, CAPABILITY_DERIVED, CAPABILITY_REVOKED
400-499	Security Events	SECURITY_ALERT, SECURITY_BREACH, CAPABILITY_VIOLATION, INTEGRITY_CHECK_FAILED
500-599	IPC Events	IPC_CHANNEL_CREATED, IPC_CHANNEL_CLOSED, IPC_MESSAGE_DROPPED
1000+	Module Events	Each module gets a 100-event range starting at MODULE_EVENT_BASE

9.2 Event Struct (64 bytes, cache-line aligned)

Field	Type	Description
m_eType	EventType (U16)	What kind of event this is
m_uSourceModuleId	U16	ModuleId of the publisher
m_uSequenceNum	U32	Monotonically increasing per-
m_uTimestampNs	U64	Nanosecond timestamp
m_payload	union (40 bytes)	Event-specific data (service, process, capability, security, or raw)

Payload Variants

Variant	Fields	Used By
m_payload.m_service	uServiceId, uModuleId, szName[26]	SERVICE_REGISTERED, SERVICE_STARTED, etc.
m_payload.m_process	uProcessId, iExitCode, uRestartCount, szReason[20]	PROCESS_SPAWNED, PROCESS_CRASHED, etc.
m_payload.m_capability	uTokenIdHigh, uTokenIdLow, uOwnerId, uPermissions, uEpoch	CAPABILITY_CREATED, CAPABILITY_REVOKED, etc.
m_payload.m_security	uSourceProcessId, uThreatLevel, szDescription[24]	SECURITY_ALERT, SECURITY_BREACH, etc.
m_payload.m_raw	arrData[40]	Module-defined events (interpret yourself)

9.3 EventBus Class

Method	Signature	Description
init	Status init(U32 ringSize = 1024)	Allocate ring buffer (must be power of 2).
shutdown	void shutdown()	Flush remaining events and deallocate.
subscribe	Result<U32> subscribe(EventType filter, EventHandler handler, name)	Register a callback. filter=NONE means subscribe to ALL events. Must be called during init phase.
publish	bool publish(const Event& event)	Push event to ring buffer. Thread-safe, lock-free. Returns false if ring full.
publishEvent	bool publishEvent(EventType type, U16 sourceModuleId)	Convenience: auto-fills timestamp and sequence number.
dispatch	U32 dispatch()	Process all pending events, call matching handlers. Called from main loop. Returns count dispatched.
pendingCount	U32 pendingCount()	Events waiting to be dispatched.
totalPublished	U64 totalPublished()	Lifetime published count.
totalDropped	U64 totalDropped()	Events dropped due to full ring.

How to Subscribe and Publish (Code Example)

```
// Subscribe to security events only
runtime.events().subscribe(
    EventType::SECURITY_ALERT,
    [](const Event& event) {
        U32 threat = event.m_payload.m_security.m_uThreatLevel;
        // handle the alert...
    },
    "my-security-handler"
);

// Subscribe to ALL events (catch-all)
runtime.events().subscribe(
```

```
    EventType::NONE,  // NONE = all events
    [](const Event& event) {
        // receives everything
    },
    "catch-all-logger"
);

// Publish an event
Event evt = Event::null();
evt.m_eType = EventType::IPC_CHANNEL_CREATED;
evt.m_uSourceModuleId = static_cast<U16>(ModuleId::IPC);
evt.m_payload.m_service.m_uServiceId = channelId;
runtime.events().publish(evt);
```

10. Process Manager (process.h)

File: include/astra/core/process.h | Implementation: src/core/process_manager.cpp

Namespace: astra::core

10.1 ProcessState Enum

State	Value	Meaning
EMPTY	0	Slot is free in the process pool
CREATED	1	Descriptor allocated, not yet spawned
SPAWNING	2	fork/clone in progress
RUNNING	3	Active and healthy
SUSPENDED	4	Paused (e.g. for checkpoint by M-04)
STOPPING	5	Graceful shutdown in progress
STOPPED	6	Exited cleanly
CRASHED	7	Exited abnormally

10.2 RestartPolicy Enum

Policy	Behaviour
NEVER	Do not restart. Let the process stay dead.
ALWAYS	Always restart on crash, regardless of exit code.
ON_FAILURE	Restart only if exit code != 0.
EXPONENTIAL	Restart with exponential backoff delays (1s, 2s, 4s, 8s...).

10.3 IsolationProfile Struct

Placeholder for M-02 (Isolation) integration. Defines the sandboxing parameters applied at process spawn time.

Field	Type	Default	Description
m_bEnableNamespaces	bool	false	Use Linux PID/mount/net/user namespaces
m_bEnableSeccomp	bool	false	Apply seccomp-BPF syscall whitelist
m_bEnableCgroups	bool	false	Apply cgroup v2 resource limits
m_uMemoryLimitMb	U32	0	Memory cap in MB (0 = unlimited)
m_uCpuSharePercent	U32	100	CPU share percentage (1-100)

10.4 ProcessConfig Struct

Field	Type	Description
m_szName	std::string	Human-readable process name
m_szBinaryPath	std::string	Path to executable
m_capToken	CapabilityToken	Capability set for this process
m_isolationProfile	IsolationProfile	Sandboxing settings (M-02 fills this)
m_eRestartPolicy	RestartPolicy	What to do when this process crashes
m_uMaxRestarts	U32	Max restart attempts (0 = infinite)
m_uParentPid	ProcessId	Parent process (0 = supervised by runtime)

10.5 ProcessManager Class

Method	Signature	Description
init	Status init(U32 maxProcesses = 256)	Allocate process descriptor pool.
shutdown	void shutdown()	Terminate all processes, wait for exit.
setEventBus	void setEventBus(EventBus*)	Connect to event bus for lifecycle events.
spawn	Result<ProcessId> spawn(const ProcessConfig&)	Spawn a new supervised process. Token must have PROC_SPAWN.
kill	Status kill(ProcessId, I32 signal)	Send signal to process. Requires PROC_KILL.
suspend	Status suspend(ProcessId)	Pause a running process (for M-04 checkpoint).
resume	Status resume(ProcessId)	Resume a suspended process.
lookupProcess	const ProcessDescriptor* lookupProcess(ProcessId)	Look up process by PID. Lock-free.
reapChildren	U32 reapChildren()	Check for exited children, handle restarts. Call from main loop.
activeCount	U32 activeCount()	Number of alive processes.
setIsolationHook	void setIsolationHook(IsolationHook)	M-02 registers this to apply namespace/seccomp/cgroup at spawn.
setProcessEventCallback	void setProcessEventCallback(cb)	M-09 registers this for eBPF tracepoints.

11. Core Runtime (runtime.h)

File: include/astra/core/runtime.h | Implementation: src/core/runtime.cpp

Namespace: astra::core

11.1 RuntimeConfig Struct

Field	Type	Default	Description
m_szInstanceName	std::string	"astra-default"	Name of this runtime instance
m_uMaxServices	U32	64	Max services in the registry
m_uMaxProcesses	U32	256	Max supervised processes
m_uMaxCapabilities	U32	4096	Max capability tokens in the pool
m_uEventRingSize	U32	1024	Event bus ring buffer size (power of 2)
m_bEnableSecurityLog	bool	true	Enable security event logging
m_bVerbose	bool	false	Enable verbose output

11.2 Runtime Class

Method	Signature	Description
init	Status init(const RuntimeConfig&)	Initialise all 6 subsystems in order. Creates root capability token.
run	Status run()	Enter main event loop. Blocks calling thread until shutdown.
shutdown	Status shutdown()	Tear down all subsystems in reverse order. Secure-wipes tokens.
state	RuntimeState state()	Current lifecycle state (UNINITIALIZED, INITIALIZING, RUNNING, etc.)
instanceName	const string& instanceName()	Runtime instance name from config.
capabilities	CapabilityManager& capabilities()	Access the capability subsystem.
services	ServiceRegistry& services()	Access the service registry.
events	EventBus& events()	Access the event bus.
processes	ProcessManager& processes()	Access the process manager.
requestShutdown	void requestShutdown()	Signal shutdown from any thread. Async-signal-safe.
isShutdownRequested	bool isShutdownRequested()	Check if shutdown has been requested.
rootCapability	const CapabilityToken& rootCapability()	The "god" token. Only runtime holds this.

Minimal Boot Example

```
#include <astra/core/runtime.h>
using namespace astra::core;

int main()
{
    Runtime runtime;
    RuntimeConfig config = RuntimeConfig::defaultConfig();
```

```
    config.m_szInstanceName = "my-astra-instance";

    auto status = runtime.init(config);
    if (!status.has_value()) { /* handle error */ }

    // Derive a token for your service
    auto tokenResult = runtime.capabilities().derive(
        runtime.rootCapability(),
        Permission::SVC_REGISTER | Permission::IPC_SEND | Permission::IPC_RECV,
        1  // owner ID
    );

    // Register your service
    MyService myService(runtime);
    runtime.services().registerService(&myService, tokenResult.value());

    // Enter main loop (blocks until shutdown)
    runtime.run();
    runtime.shutdown();
    return 0;
}
```

12. Logger (logger.h)

File: include/astra/core/logger.h | Implementation: src/core/logger.cpp

Namespace: astra::core

12.1 LogLevel Enum

Level	Value	Console Colour	When to Use
TRACE	0	Grey	Function entry/exit, loop iterations
DEBUG	1	Cyan	Variable dumps, state transitions
INFO	2	Green	Startup, shutdown, config changes
WARN	3	Yellow	Retry, fallback, degraded mode
ERROR	4	Red	Operation aborted, data loss risk
FATAL	5	Bold Red	Unrecoverable, runtime must terminate
OFF	6	-	Disable all logging

12.2 Logger Class

Method	Signature	Description
initialise	bool initialise(moduleName, filePath, minLevel, alsoConsole)	Open/create log file. Auto-creates parent directories.
shutdown	void shutdown()	Flush and close. Safe to call multiple times.
isInitialised	bool isInitialised()	Check if ready to accept writes.
setMinLevel	void setMinLevel(LogLevel)	Change minimum log level at runtime.
getMinLevel	LogLevel getMinLevel()	Read current minimum log level.
log	void log(level, file, line, message)	Core write function. Normally called through macros.

12.3 How to Set Up a Logger (3 Steps)

Step 1: DECLARE in a header (extern declaration)

```
// file: include/astra/ipc/ipc_logger.h
#pragma once
#include <astra/core/logger.h>
ASTRA_DECLARE_LOGGER(g_logIpc);
```

Step 2: DEFINE in exactly ONE .cpp (creates the global instance)

```
// file: src/ipc/ipc_channel.cpp
#include "ipc_logger.h"
ASTRA_DEFINE_LOGGER(g_logIpc);
```

Step 3: INITIALISE at module startup

```
g_logIpc.initialise("ipc", "logs/ipc.log", astra::core::LogLevel::DEBUG, true);
```

12.4 Logging Macros

```
LOG_TRACE(g_logIpc, "Entering sendMessage()");
LOG_DEBUG(g_logIpc, "Payload size: " << uBytes << " bytes");
LOG_INFO(g_logIpc,  "Channel " << uChannelId << " created");
LOG_WARN(g_logIpc,  "Ring buffer " << uPctFull << "% full");
LOG_ERROR(g_logIpc, "HMAC mismatch on msg seq " << uSeq);
LOG_FATAL(g_logIpc, "Shared memory corrupted");
```

Output Format

```
[2026-03-17 14:22:03.123456] [ipc    ] [INFO ] [ipc_channel.cpp:42] Channel 7
created
```

12.5 Existing Module Loggers

Global Variable	Module	Log File	Defined In
g_logCore	Runtime lifecycle	logs/core.log	runtime.cpp
g_logCapability	Capability operations	logs/capability.log	capability.cpp
g_logSvcRegistry	Service registration	logs/svc_registry.log	service_registry.cpp
g_logEventBus	Event publish/subscribe	logs/event_bus.log	event_bus.cpp
g_logProcMgr	Process management	logs/proc_mgr.log	process_manager.cpp
g_logDemo	Demo service	logs/demo.log	demo_service.cpp
g_logMain	Main entry point	logs/main.log	main.cpp
g_logDemoMain	Demo entry point	logs/demo_main.log	demo_main.cpp

13. Demo Service (demo_service.h)

File: include/astra/core/demo_service.h | Implementation: src/core/demo_service.cpp

Entry point: src/demo_main.cpp | Binary: astra_demo

DemoService is a concrete IService implementation that exercises every Core subsystem. It is the reference implementation that every team member should study to understand how their module should integrate with M-01.

Demo Sections

Method	What It Demonstrates
demoCapabilities()	Token create, derive, validate, escalation block, cascading revocation
demoServiceRegistry()	Service lookup by name/ID, capability-gated registration denial
demoEventBus()	Subscribe with filter + catch-all, publish events, dispatch and verify delivery
demoProcessManager()	Spawn process descriptors with ALWAYS and EXPONENTIAL restart policies
demoSecurityViolation()	Restricted token checked against 8 permissions, escalation blocked

How to Run the Demo

```
# Build
cmake --preset debug
cmake --build build/

# Run
./build/astra_demo
```

14. Build Instructions

Prerequisites

- GCC 13+ or Clang 17+ (C++23 required for std::expected)
- CMake 3.28+
- Linux x86-64 only
- NASM (for M-21 ASM Core assembly routines)

Build Commands

```
# Debug build (default)
cmake --preset debug
cmake --build build/

# Release build (optimised)
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build/

# With sanitizers (ASan + UBSan)
cmake -B build -DASTRA_ENABLE_SANITIZERS=ON
cmake --build build/

# With thread sanitizer
cmake -B build -DASTRA_ENABLE_TSAN=ON
cmake --build build/
```

Build Targets

Target	Description	Output
astra_runtime	Main runtime executable	build/astra_runtime
astra_demo	Interactive demo of Core subsystems	build/astra_demo
astra_core	Static library for all Core module code	build/src/core/libastra_core.a
astra_asm_core	Static library for M-21 ASM crypto	build/src/asm_core/ libastra_asm_core.a

15. Module Roadmap & Team Assignments

Module	Assigned To	Status	Key Classes to Build
M-01 Core	Shiva	COMPLETE	Runtime, CapabilityManager, ServiceRegistry, EventBus, ProcessManager, Logger
M-02 Isolation	Harini & Srinivasan	TO BUILD	NamespaceManager, SeccompFilter, CgroupController, IsolationEnforcer
M-03 IPC	Srivatsan & Laakhi	TO BUILD	Channel, RingBuffer, HmacSigner, ChannelManager, MessageHeader
M-09 eBPF	Manoj & Dhruv	TO BUILD	EbpfLoader, EbpfMapReader, MetricsAggregator, AnomalyDetector, TelemetryExporter
M-21 ASM Core	Shiva	STUB	asm_rdrand64, asm_secure_wipe (NASM implementations pending)

Integration Checklist for New Modules

Every new module must:

- Implement the IService interface (name, moduleId, onInit, onStart, onStop, isHealthy)
- Register with the ServiceRegistry using a derived capability token
- Subscribe to relevant events on the EventBus during onInit()
- Create a per-module Logger instance (ASTRA_DEFINE_LOGGER) with its own log file
- Validate capability tokens before performing any gated operation
- Use Result<T> / Status for all error-returning functions

- Follow the m_iVariableName naming convention strictly
- Never call another module directly; use IPC or event bus for cross-module communication